Rutas, vistas y plantillas en Django

Presentación

En este trabajo vas a aprender cómo Django responde cuando alguien escribe una dirección en el navegador. Recorreremos el camino completo de una petición: la **ruta** que la recibe, la **vista** que decide qué hacer y la **plantilla** que muestra el resultado. Al final construirás un catálogo de libros y le agregarás una función propia.

Al terminar podrás:

1. Explicar con tus palabras el patrón MTV de Django.
2. Crear rutas, con y sin parámetros, y organizarlas por app con `include`.
3. Escribir vistas como funciones y como clases.
4. Mostrar datos en plantillas HTML usando variables, `for`, `if` y herencia.
5. Detectar y corregir los errores más comunes.

Preparación del proyecto

Todo el trabajo usa el mismo proyecto. Necesitas Python 3.10 o superior. Los comandos son para PowerShell en Windows.

```
1. mkdir catalogo_django
2. cd catalogo_django
3. python -m venv catalogo   # crea un entorno virtual: una instalación de Python
   solo para este Proyecto
4. \catalogo\Scripts\Activate        # actívalo; el prompt debe empezar con
   (venv)
5. pip install Django
6. django-admin startproject config .  # el punto crea el proyecto en esta misma
   carpeta
7. python manage.py startapp catalogo  # una app es un módulo con una función
   concreta del sitio
```

- Registra la app en `config/settings.py`, dentro de la lista `INSTALLED_APPS`:

```
INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",
    "catalogo",  # nuestra app
]
```

Prueba que todo funciona con `python manage.py runserver` y abre `http://127.0.0.1:8000/`. Debes ver la página de bienvenida de Django (un cohete).

Parte 1: El patrón MTV

Un **patrón** es una forma probada de organizar el código. Django usa el patrón **MTV** (Modelo-Plantilla-Vista), que reparte el trabajo en tres piezas para que cada una tenga una sola responsabilidad:

Pieza	Archivo	Responsabilidad	Analogía en un restaurante
Modelo	`models.py`	Define y guarda los datos en la base de datos	La despensa
Plantilla (Template)	archivos `.html`	Define cómo se ve la página	El plato servido
Vista	`views.py`	Recibe la petición, busca los datos y elige la plantilla	El cocinero

Falta una pieza que no aparece en la sigla pero es la puerta de entrada: la **ruta** (`urls.py`), que es como el mesero que recibe el pedido y lo lleva al cocinero correcto.

El recorrido de una petición

```
Navegador pide /libros/3/
        │
        ▼
urls.py   → busca qué ruta coincide y llama a su vista
        │
        ▼
views.py  → busca el libro 3 (en el modelo o en una lista)
        │
        ▼
plantilla → arma el HTML con los datos del libro
        │
        ▼
Navegador recibe la página
```

En este trabajo nos concentramos en la ruta, la vista y la plantilla. Para no depender de una base de datos, los datos estarán en una lista de Python; cuando aprendas modelos, solo cambiará la línea donde la vista obtiene los datos.

Parte 2: Rutas (`urls.py`) del proyecto

Una **ruta** relaciona una dirección (por ejemplo `/libros/`) con la vista que debe responderla. Django las lee de arriba abajo en la lista `urlpatterns` y usa la primera que coincide.

2.1 Una ruta sencilla

Cada ruta se crea con `path()`, que recibe tres datos: la dirección, la vista y un nombre.

```
path("libros/", views.lista_libros, name="lista_libros")
#     │          │                    └── alias para referirse a la ruta sin escribir
la dirección
#     │          └── función que responde
#     └── dirección, sin barra inicial y con barra final
```

2.2 Rutas con parámetros

Una ruta puede recibir **parámetros**: datos que viajan dentro de la propia dirección. Por ejemplo, para ver la información de un producto por su **id,** esperaríamos una dirección como `dominio/productos/1/`. En esa solicitud, el número **1** es el parámetro que la ruta debe capturar. Se escribe en la ruta dentro del path entre `< >`, indicando su tipo con un **convertidor**:

```
# urls.py del Proyecto
path("productos/<int:id>/", views.ver_producto, name="ver_producto")
#                └── convertidor int: solo acepta números enteros y entrega el valor
como int
```

La vista asociada a la ruta debe recibir ese parámetro, después de `request` y con el **mismo nombre** que tiene en la ruta:

```
# views.py de la app

from django.http import HttpResponse


def ver_producto(request, id):
    return HttpResponse(f"Producto con ID: {id}")
```

Dirección que se abre	Resultado
`/productos/1/`	"Producto con ID: 1"
`/productos/25/`	"Producto con ID: 25"
`/productos/abc/`	Error 404: el convertidor `int` no acepta texto y Django ni siquiera llama a la vista

El nombre `id` funciona, pero en el catálogo usaremos nombres más descriptivos, como `libro_id`: dicen a qué se refiere el número y no ocultan la función `id()` que Python ya trae. Lo importante es que el nombre coincida en la ruta y en la vista.

Estos son los convertidores más usados:

Convertidor	Acepta	Ejemplo de dirección
`<int:libro_id>`	Números enteros	`/libros/3/`
`<str:genero>`	Texto sin `/`	`/libros/genero/novela/`

Convertidor	Acepta	Ejemplo de dirección
`<slug:titulo>`	Letras, números, `-` y `_`	`/libros/cien-anos/`

El valor capturado llega a la vista como un argumento con ese mismo nombre.

Urls.py

```
path("libros/<int:libro_id>/", views.detalle_libro, name="detalle_libro")

# La vista se define como: def detalle_libro(request, libro_id) views.py
```

2.3 Dividir las rutas por app con `include`

Un proyecto puede tener varias apps. En lugar de poner todas las rutas en un solo archivo, cada app tiene su propio `urls.py` y el proyecto lo **incluye**.

Crea el archivo `catalogo/urls.py` (Django no lo crea solo):

```
# catalogo/urls.py

from django.urls import path

from . import views

app_name = "catalogo"  # espacio de nombres: las rutas se llamarán "catalogo:..."


urlpatterns = [
    path("", views.inicio, name="inicio"),
]
```

Y conéctalo en el `urls.py` del proyecto:

```
# config/urls.py

from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path("admin/", admin.site.urls),
    path("", include("catalogo.urls")),  # las rutas de la app quedan en la raíz del
sitio
]
```

Si escribieras

`path("tienda/", include("catalogo.urls"))`, todas las rutas de la app empezarían por `/tienda/`
Gracias a `app_name`,

otra app también podría tener una ruta llamada `inicio` sin chocar con la nuestra.

2.4 Usar el nombre de la ruta

Nunca escribas la dirección a mano en vistas ni plantillas: usa el nombre. Si un día cambias la dirección en `urls.py`, todos los enlaces se actualizan solos.

Dónde	Cómo
En una plantilla	`<a href="{% url 'catalogo:detalle_libro' 3 %}">`
En una vista	`return redirect("catalogo:detalle_libro", libro_id=3)`
En Python, para obtener el texto de la dirección	`reverse("catalogo:detalle_libro", args=[3])` → `'/libros/3/'`

Todavía no puedes ejecutar el servidor: `views.inicio` no existe. Lo resolvemos en la siguiente parte.

Parte 3: Vistas (`views.py`)

Una **vista** es una función (o una clase) que recibe la petición del navegador y devuelve una **respuesta**: una página, un texto, un error o una redirección.

3.1 Vistas como funciones

Toda vista de función recibe como primer argumento `request`, un objeto con la información de la petición. Los más usados:

Atributo	Qué contiene	Ejemplo
`request.method`	El método HTTP	`"GET"` al abrir una página, `"POST"` al enviar un formulario
`request.path`	La dirección pedida	`"/libros/"`
`request.GET`	Datos enviados en la dirección, después de `?`	En `/libros/?genero=novela`, `request.GET.get("genero")` vale `"novela"`
`request.POST`	Datos enviados por un formulario con `method="post"`	`request.POST.get("nombre")`

La vista más sencilla responde un texto con `HttpResponse`:

```
# catalogo/views.py
```

```
from django.http import HttpResponse


def inicio(request):
    return HttpResponse("¡Bienvenido al catálogo!")
```

Ejecuta `python manage.py runserver` y abre `http://127.0.0.1:8000/`. Ya tienes ruta y vista funcionando.

Cuando la ruta tiene parámetros, la vista los recibe después de `request`:

```
def saludo(request, nombre):   # ruta: urls.py path("saludo/<str:nombre>/", ...)
return HttpResponse(f"Hola, {nombre}")
```

En la práctica casi nunca se responde con texto suelto. Lo normal es usar `render()`, que veremos con las plantillas, y estas herramientas:

Herramienta	Para qué sirve
`render(request, plantilla, contexto)`	Devuelve una página HTML armada con una plantilla y datos
`redirect("nombre_de_ruta")`	Envía al navegador a otra página
`raise Http404("mensaje")`	Responde "página no encontrada" cuando algo no existe

3.2 Introducción a las vistas basadas en clases

Django también permite escribir vistas como **clases** (en inglés, *class-based views* o CBV). Una clase es una plantilla de código que se puede reutilizar y extender. Django trae clases listas para tareas repetidas; la más sencilla es `TemplateView`, que solo muestra una plantilla:

```
from django.views.generic import TemplateView


class AcercaView(TemplateView):
    template_name = "catalogo/acerca.html"  # la plantilla que mostrará
```

En `urls.py`, una clase se conecta con `.as_view()`, que la convierte en una función que Django puede llamar:

```
path("acerca/", views.AcercaView.as_view(), name="acerca")
```

Para enviar datos a la plantilla desde una clase, se sobrescribe el método `get_context_data`:

```
class AcercaView(TemplateView):
    template_name = "catalogo/acerca.html"

    def get_context_data(self, **kwargs):
        contexto = super().get_context_data(**kwargs)  # contexto que Django ya
prepara
        contexto["version"] = "1.0"                    # nuestro dato adicional
        return context
```

¿Función o clase? Para empezar, las funciones son más fáciles de leer porque todo está a la vista. Las clases ahorran código cuando las tareas se repiten (listados, formularios, detalles). En este trabajo usaremos sobre todo funciones y una clase para conocerlas.

Parte 4: Plantillas

Una **plantilla** es un archivo HTML con huecos que Django rellena con datos. Esos huecos se escriben en el **lenguaje de plantillas de Django** (DTL, por *Django Template Language*).

4.1 Estructura de carpetas

Django busca las plantillas dentro de una carpeta `templates` en cada app. Por convención, adentro se crea otra carpeta con el nombre de la app:

```
catalogo/
├── templates/
│   └── catalogo/          ← se repite el nombre de la app a propósito
│       ├── base.html
│       └── inicio.html
├── urls.py
└── views.py
```

Ese nombre repetido evita confusiones: si otra app tiene un `inicio.html`, cada una se pide por su ruta completa, como `"catalogo/inicio.html"`. La carpeta debe llamarse exactamente `templates`, en plural y en minúscula.

4.2 Pasar datos con el contexto

El **contexto** es un diccionario que la vista entrega a la plantilla. Cada clave se convierte en una variable dentro del HTML.

```
# catalogo/views.py

from django.shortcuts import render


def inicio(request):
    contexto = {"nombre_sitio": "Catálogo ADSO", "total": 6}
    return render(request, "catalogo/inicio.html", contexto)
```

4.3 Variables y etiquetas

Sintaxis	Qué hace	Ejemplo
`{{ variable }}`	Muestra un valor	`{{ nombre_sitio }}`
`{{ objeto.campo }}`	Muestra un dato de un diccionario u objeto	`{{ libro.titulo }}`
`{{ valor|filtro }}`	Transforma un valor con un **filtro**	`{{ libros|length }}`, `{{ titulo|upper }}`
`{% for x in lista %} ... {% empty %} ... {% endfor %}`	Repite un bloque por cada elemento; `empty` se muestra si la lista está vacía	Listar libros
`{% if condicion %} ... {% else %} ... {% endif %}`	Muestra un bloque solo si se cumple la condición	Disponible / Prestado
`{% url 'nombre' parametro %}`	Escribe la dirección de una ruta	Enlaces
`{# comentario #}`	Comentario que no llega al navegador	Notas

Regla práctica: `{{ }}` **muestra** algo; `{% %}` **hace** algo.

4.4 Herencia con `extends` y `block`

Casi todas las páginas comparten encabezado, menú y pie. Para no repetirlos, se crea una plantilla **base** con huecos llamados **bloques**, y las demás la **heredan** y solo rellenan esos huecos.

```
<!-- catalogo/templates/catalogo/base.html -->
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>{% block titulo %}Catálogo{% endblock %}</title>
</head>
<body>
    <nav>
        <a href="{% url 'catalogo:inicio' %}">Inicio</a>
    </nav>
    <main>
        {% block contenido %}{% endblock %}  {# hueco que cada página rellena #}
    </main>
    <footer>Proyecto de clase ADSO</footer>
</body>
</html>
```

```
<!-- catalogo/templates/catalogo/inicio.html -->
{% extends "catalogo/base.html" %}  {# siempre debe ser la primera línea #}

{% block titulo %}Inicio{% endblock %}

{% block contenido %}
    <h1>Bienvenido a {{ nombre_sitio }}</h1>
    {% if total > 0 %}
        <p>Tenemos {{ total }} libros en el catálogo.</p>
    {% else %}
        <p>Todavía no hay libros.</p>
    {% endif %}
{% endblock %}
```

Recarga la página: ahora ves el menú, el título y el pie, y solo escribiste el contenido propio de la página de inicio.

Parte 5: Mini-proyecto integrador "Catálogo de libros"

Ahora unimos todo. El catálogo tendrá cuatro páginas:

Dirección	Nombre de la ruta	Vista	Plantilla
`/`	`inicio`	`inicio` (función)	`inicio.html`
`/libros/` y `/libros/?genero=novela`	`lista_libros`	`lista_libros` (función)	`lista.html`
`/libros/<id>/`	`detalle_libro`	`detalle_libro` (función)	`detalle.html`
`/acerca/`	`acerca`	`AcercaView` (clase)	`acerca.html`

Paso 1. Los datos

Crea `catalogo/datos.py`. Los libros y autores son ficticios.

```
# catalogo/datos.py
# Datos de ejemplo. Más adelante vendrán de un modelo y una base de datos.


LIBROS = [
    {"id": 1, "titulo": "El río de las palabras", "autor": "Marta Ríos", "anio":
1998, "genero": "novela", "disponible": True},
    {"id": 2, "titulo": "Noches de lluvia", "autor": "Julián Ortega", "anio": 2005,
"genero": "novela", "disponible": False},
    {"id": 3, "titulo": "Python desde cero", "autor": "Ana Ruiz", "anio": 2021,
"genero": "tecnologia", "disponible": True},
    {"id": 4, "titulo": "El universo en tu bolsillo", "autor": "Carlos Vega", "anio":
2012, "genero": "ciencia", "disponible": True},
    {"id": 5, "titulo": "Cuentos del Magdalena", "autor": "Marta Ríos", "anio": 1987,
"genero": "novela", "disponible": True},
    {"id": 6, "titulo": "Django paso a paso", "autor": "Laura Méndez", "anio": 2024,
"genero": "tecnologia", "disponible": False},
```

```
]


def buscar_libro(libro_id):
    """Devuelve el libro con ese id, o None si no existe."""
    for libro in LIBROS:
        if libro["id"] == libro_id:
            return libro
    return None
```

Paso 2. Las rutas

```
# catalogo/urls.py
from django.urls import path

from . import views

app_name = "catalogo"

urlpatterns = [
    path("", views.inicio, name="inicio"),
    path("libros/", views.lista_libros, name="lista_libros"),
    path("libros/<int:libro_id>/", views.detalle_libro, name="detalle_libro"),
    path("acerca/", views.AcercaView.as_view(), name="acerca"),
]
```

`config/urls.py` queda igual que en la parte 2.

Paso 3. Las vistas

Reemplaza todo el contenido de `catalogo/views.py`:

```
# catalogo/views.py
from django.http import Http404
from django.shortcuts import render
from django.views.generic import TemplateView

from .datos import LIBROS, buscar_libro


def inicio(request):
    contexto = {"nombre_sitio": "Catálogo ADSO", "total": len(LIBROS)}
    return render(request, "catalogo/inicio.html", contexto)


def lista_libros(request):
    genero = request.GET.get("genero", "")  # "" si no viene ?genero= en la dirección
    if genero:
        libros = [libro for libro in LIBROS if libro["genero"] == genero]
    else:
        libros = LIBROS
    return render(request, "catalogo/lista.html", {"libros": libros, "genero":
genero})
```

```
def detalle_libro(request, libro_id):
    libro = buscar_libro(libro_id)
    if libro is None:
        raise Http404("El libro no existe")
    return render(request, "catalogo/detalle.html", {"libro": libro})


class AcercaView(TemplateView):
    template_name = "catalogo/acerca.html"

    def get_context_data(self, **kwargs):
        contexto = super().get_context_data(**kwargs)
        contexto["version"] = "1.0"
        return contexto
```

Paso 4. Las plantillas

Actualiza el menú de `base.html` para enlazar las tres secciones:

```
    <nav>
        <a href="{% url 'catalogo:inicio' %}">Inicio</a> |
        <a href="{% url 'catalogo:lista_libros' %}">Libros</a> |
        <a href="{% url 'catalogo:acerca' %}">Acerca de</a>
    </nav>
```

`inicio.html` queda como en la parte 4.

Crea las otras tres:

```
<!-- catalogo/templates/catalogo/lista.html -->


{% extends "catalogo/base.html" %}

{% block titulo %}Libros{% endblock %}

{% block contenido %}
    <h1>Libros{% if genero %} de {{ genero }}{% endif %}</h1>

    <p>
        Filtrar:
        <a href="{% url 'catalogo:lista_libros' %}">Todos</a> |
        <a href="{% url 'catalogo:lista_libros' %}?genero=novela">Novela</a> |
        <a href="{% url 'catalogo:lista_libros' %}?genero=tecnologia">Tecnología</a>
|
        <a href="{% url 'catalogo:lista_libros' %}?genero=ciencia">Ciencia</a>
    </p>

    <ul>
        {% for libro in libros %}
            <li>
                <a href="{% url 'catalogo:detalle_libro' libro.id %}">{{ libro.titulo
}}</a>
```

```
                ({{ libro.anio }})
                {% if libro.disponible %}✅ Disponible{% else %}❌ Prestado{% endif
%}
            </li>
        {% empty %}
            <li>No hay libros de ese género.</li>
        {% endfor %}
    </ul>

    <p>Total: {{ libros|length }}</p>
{% endblock %}
```

```
<!-- catalogo/templates/catalogo/detalle.html -->



{% extends "catalogo/base.html" %}

{% block titulo %}{{ libro.titulo }}{% endblock %}

{% block contenido %}
    <h1>{{ libro.titulo }}</h1>
    <p><strong>Autor:</strong> {{ libro.autor }}</p>
    <p><strong>Año:</strong> {{ libro.anio }}</p>
    <p><strong>Género:</strong> {{ libro.genero|capfirst }}</p>
    <p><strong>Estado:</strong> {{ libro.disponible|yesno:"Disponible,Prestado"
}}</p>
    <a href="{% url 'catalogo:lista_libros' %}">← Volver al listado</a>
{% endblock %}
```

```
<!-- catalogo/templates/catalogo/acerca.html -->
{% extends "catalogo/base.html" %}

{% block titulo %}Acerca de{% endblock %}

{% block contenido %}
    <h1>Acerca del catálogo</h1>
    <p>Proyecto de clase para practicar rutas, vistas y plantillas. Versión {{
version }}.</p>
{% endblock %}
```

Paso 5. Comprueba

Abre	Debes ver
`/`	La bienvenida con "Tenemos 6 libros"
`/libros/`	Los 6 libros y "Total: 6"
`/libros/?genero=novela`	Solo las 3 novelas
`/libros/?genero=poesia`	"No hay libros de ese género."

Abre	Debes ver
`/libros/3/`	El detalle de "Python desde cero"
`/libros/99/`	Página de error 404
`/acerca/`	La página hecha con la vista de clase

Errores frecuentes y cómo resolverlos

Equivocarse es parte de aprender. Lo importante es **leer el mensaje de error**: Django casi siempre indica el archivo, la línea y la causa. Lee primero la última línea del mensaje; es la que dice qué pasó.

Síntoma	Causa	Cómo detectarlo	Cómo corregirlo
`TemplateDoesNotExist` aunque el archivo existe	La app no está en `INSTALLED_APPS`, entonces Django no busca en su carpeta `templates`	Revisa la lista en `config/settings.py`	Agrega `"catalogo"` a `INSTALLED_APPS`
`TemplateDoesNotExist` con la app registrada	La carpeta se llama `template` o `Templates`, falta la subcarpeta `catalogo/` o el nombre del archivo no coincide	La página de error muestra en qué carpetas buscó Django	Usa `catalogo/templates/catalogo/archivo.html`
Error 404 "Page not found"	La dirección escrita no coincide con ninguna ruta (falta la barra final, hay una letra distinta)	La página de error lista todas las rutas que Django conoce	Compara letra por letra con `urls.py`
`NoReverseMatch`	En `{% url %}` o `redirect()` el nombre está mal escrito, falta el prefijo `catalogo:` o falta el parámetro	El mensaje dice qué nombre buscó	Copia el nombre exacto de `name=` y agrega el parámetro si la ruta lo pide
`AttributeError: module 'catalogo.views' has no attribute ...`	`urls.py` nombra una vista que no existe o está mal escrita	El mensaje dice qué nombre faltó	Crea la vista o corrige el nombre

Síntoma	Causa	Cómo detectarlo	Cómo corregirlo
`TypeError: ... got an unexpected keyword argument`	El parámetro de la ruta y el de la vista tienen nombres distintos (`<int:id>` y `def detalle(request, libro_id)`)	El mensaje nombra el argumento	Usa el mismo nombre en los dos lugares
Error 403 "CSRF verification failed" al enviar un formulario	Falta `{% csrf_token %}` dentro del `<form method="post">`	Aparece solo al enviar con POST	Agrega `{% csrf_token %}` como primera línea dentro del `<form>`
`TemplateSyntaxError`	Una etiqueta sin cerrar (`{% for %}` sin `{% endfor %}`) o `{% extends %}` que no está en la primera línea	El mensaje señala la línea de la plantilla	Cierra cada etiqueta y pon `extends` arriba
La página sale en blanco donde debía haber un dato	La variable de la plantilla no coincide con la clave del contexto. Django no da error: muestra vacío	Compara `{{ nombre }}` con las claves del diccionario	Usa exactamente el mismo nombre
Los cambios no se ven	El archivo no se guardó o el servidor se detuvo por un error	Mira la terminal donde corre `runserver`	Guarda (Ctrl+S) y revisa la terminal

Ojo: ejecuta `python manage.py check` después de cada cambio en `settings.py` o `urls.py`. Si responde `System check identified no issues`, la configuración está bien.

Ejercicios generales

Todos se hacen sobre el mini-proyecto y van de menor a mayor dificultad. Cada uno agrega algo nuevo sin romper lo anterior. Antes de pasar al siguiente, comprueba en el navegador que funciona.

Ejercicio 1. Tu primera respuesta (ruta + vista). Crea la ruta `/hola/`, con nombre `hola`, cuya vista responda con `HttpResponse` el texto "Hola desde Django".

Ejercicio 2. Ruta con parámetro. Crea la ruta `/cuadrado/<numero>/`, con nombre `cuadrado`, que solo acepte enteros y responda, por ejemplo, "El cuadrado de 7 es 49". Comprueba qué pasa si escribes `/cuadrado/siete/` y explica por qué.

Ejercicio 3. Página con herencia. Crea la página `/contacto/` usando una vista de clase `TemplateView`. La plantilla debe heredar de `base.html` y mostrar un correo y un horario de atención. Agrega el enlace en el menú usando `{% url %}`.

Ejercicio 4. Contexto y condicionales. En la página de inicio, muestra además cuántos libros están disponibles y cuántos prestados. La vista calcula los dos números; la plantilla los muestra. Si no hay libros prestados, muestra "¡Todos los libros están en la biblioteca!".

Ejercicio 5. Recorrer datos calculados. Crea `/autores/`, con nombre `autores`, que muestre la lista de autores sin repetir, ordenada alfabéticamente, y junto a cada uno cuántos libros tiene. Usa `{% for %}` en la plantilla.

Ejercicio 6. Formulario con POST. Crea `/sugerir/`, con nombre `sugerir`, con un formulario que pida el título de un libro que el aprendiz quisiera en la biblioteca. Al enviarlo con `POST`, la misma página muestra "Gracias, tendremos en cuenta: (título)". Si se envía vacío, muestra "Escribe un título". No olvides la protección CSRF.

Ejercicios individuales

Cada aprendiz tiene un ejercicio propio que agrega una función nueva al catálogo. Todos usan los datos de `datos.py` y la plantilla `base.html`; ningún ejercicio necesita base de datos.

Condiciones para todos: la ruta tiene nombre y está en `catalogo/urls.py`; la plantilla hereda de `base.html`; todo enlace se construye con `{% url %}`; y la página se puede abrir desde el menú o desde otra página del catálogo.

Consigna de exposición (5 minutos por aprendiz). Además de la consigna propia de cada ejercicio, toda exposición debe: mostrar la página funcionando; mostrar la ruta en `urls.py` y explicar cada parte del `path()`; recorrer la vista línea por línea, diciendo qué datos pone en el contexto; señalar en la plantilla dónde usó `{{ }}`, `{% for %}` o `{% if %}` y cómo hereda de la base; y contar una dificultad que encontró, qué mensaje de error vio y cómo la resolvió.

Aprendiz Isabella. Libros disponibles

Crea `/libros/disponibles/`, con nombre `disponibles`, que muestre solo los libros disponibles, con el total al final. Cada título enlaza a su detalle. Agrega el enlace en el menú.

Exposición: explica cómo la vista filtra la lista antes de enviarla y por qué es mejor filtrar en la vista que con un `{% if %}` en la plantilla.

Aprendiz Luis Miguel. Libros por género con parámetro de texto

Crea `/libros/genero/<genero>/`, con nombre `por_genero`, que muestre los libros de ese género usando un convertidor `str`. Si no hay libros de ese género, responde 404. En la página de detalle, convierte el género en un enlace a esta ruta.

Exposición: compara tu ruta con el filtro `?genero=` del mini-proyecto: en qué se diferencian un parámetro de ruta y un parámetro de `request.GET`, y cuándo conviene cada uno.

Aprendiz Santiago. Libros publicados desde un año

Crea `/libros/desde/<anio>/`, con nombre `desde_anio`, que solo acepte enteros y muestre los libros publicados ese año o después, ordenados del más reciente al más antiguo. El título de la página debe decir "Libros desde (año)".

Exposición: demuestra qué pasa con `/libros/desde/abc/` y explica qué hace el convertidor `int` antes de que se ejecute tu vista.

Aprendiz Anderson. Libro al azar

Crea `/libros/azar/`, con nombre `azar`, que elija un libro aleatorio con el módulo `random` y lo muestre **reutilizando** la plantilla `detalle.html`. Agrega en la página de inicio un botón "Sorpréndeme" que lleve a esta ruta.

Exposición: explica cómo dos vistas distintas pueden usar la misma plantilla y qué debe tener el contexto para que funcione.

Aprendiz Leonardo. Buscador por título

Crea `/buscar/`, con nombre `buscar`, con un formulario que use el método `GET` y un campo llamado `q`. La vista muestra los libros cuyo título contiene ese texto, sin importar mayúsculas. Si no se escribió nada, muestra solo el formulario; si no hay resultados, muestra "No se encontraron libros para (texto)". El campo debe conservar lo que se escribió.

Exposición: explica por qué este formulario usa `GET` y no necesita `{% csrf_token %}`, y cómo llega el texto escrito a la vista.

Aprendiz Juan Pablo. Estadísticas del catálogo

Crea `/estadisticas/`, con nombre `estadisticas`, que muestre: total de libros, disponibles, prestados, porcentaje disponible (sin decimales) y una tabla con cuántos libros hay por género. Si el porcentaje es menor al 50 %, muestra una advertencia.

Exposición: muestra el diccionario de contexto que arma tu vista y explica por qué los cálculos van en la vista y no en la plantilla.

Aprendiz Soto. Novedades con vista basada en clase

Crea `/novedades/`, con nombre `novedades`, usando una clase que herede de `TemplateView`. Debe mostrar los libros publicados desde 2010, del más reciente al más antiguo. Los datos se envían con `get_context_data`.

Exposición: compara tu clase con la vista de función `lista_libros`: qué hace `as_view()`, qué hace `get_context_data` y qué ventajas y desventajas ves en cada forma.

Aprendiz Angel. Navegación anterior y siguiente

Modifica la vista `detalle_libro` para que envíe también el libro anterior y el siguiente de la lista. En `detalle.html`, muestra los enlaces "← Anterior" y "Siguiente →" solo cuando existan: el primer libro no tiene anterior y el último no tiene siguiente.

Exposición: explica cómo calculaste los vecinos, por qué usaste `{% if %}` en la plantilla y cómo comprobaste los dos extremos.

Aprendiz Laura. Listado ordenado

Crea `/libros/orden/<campo>/`, con nombre `ordenados`, que muestre los libros ordenados por `titulo`, `anio` o `autor`, según el parámetro. Cualquier otro valor responde 404. Agrega sobre el listado tres enlaces para cambiar el orden y resalta en negrita el orden activo.

Exposición: explica cómo validaste el parámetro antes de usarlo y por qué es peligroso confiar en un valor que llega en la dirección.

Las soluciones están en el "Anexo docente: soluciones de los ejercicios individuales".

Objetivos del trabajo

Objetivo	Logrado cuando el aprendiz…
1. Patrón MTV	Explica qué hace la ruta, la vista y la plantilla en una página de su proyecto, y en qué orden intervienen
2. Rutas	Crea rutas con nombre, usa el convertidor adecuado (`int`, `str`), organiza las rutas de la app con `include` y `app_name`, y no escribe direcciones a mano
3. Vistas	Escribe vistas de función que reciben los parámetros de la ruta, usan `request` cuando hace falta y responden con `render`, `redirect` o `Http404`; y escribe al menos una vista con `TemplateView`
4. Plantillas	Ubica las plantillas en `app/templates/app/`, hereda de `base.html` con `extends` y `block`, y usa correctamente `{{ }}`, `{% for %}`, `{% if %}` y `{% url %}`
5. Errores	Lee un mensaje de error, identifica el archivo y la causa, y lo corrige sin ayuda o con una pista mínima

Ejercicio individual (70 %)

Criterio	Logrado	En proceso	No logrado
Funcionamiento (25 %)	La página funciona en todos los casos pedidos, incluidos los	Funciona en el caso normal pero falla en algún caso límite	No funciona o da error

Criterio	Logrado	En proceso	No logrado
	casos límite (404, lista vacía, extremos)		
Ruta (15 %)	Ruta con nombre, convertidor correcto y registrada en el `urls.py` de la app	Funciona pero sin nombre o con un convertidor inadecuado	La ruta no responde
Vista (20 %)	La lógica está en la vista, el contexto es claro y los nombres de variables son descriptivos	La lógica funciona pero parte está en la plantilla o es confusa	La vista no prepara los datos pedidos
Plantilla (15 %)	Hereda de la base, usa las etiquetas adecuadas y todos los enlaces con `{% url %}`	Hereda de la base pero repite HTML o escribe alguna dirección a mano	No hereda o no muestra los datos
Integración (15 %)	La página es accesible desde el menú u otra página, y el resto del catálogo sigue funcionando	Funciona aislada, sin enlace desde el sitio	Rompe otras páginas

Exposición oral (30 %)

Criterio	Logrado	En proceso	No logrado
Recorrido técnico (40 %)	Explica en orden ruta → vista → contexto → plantilla, señalando el código	Explica las piezas pero sin conectarlas	Solo muestra la página
Vocabulario (20 %)	Usa con precisión ruta, vista, contexto, plantilla, convertidor, herencia	Usa los términos con algunas imprecisiones	No usa los términos del trabajo
Consigna propia (20 %)	Responde completa la pregunta específica de su ejercicio	La responde de forma parcial	No la aborda
Dificultad y solución (20 %)	Describe un error real, el mensaje que vio y cómo lo resolvió	Menciona una dificultad sin explicar la solución	No menciona dificultades